

All upcoming events → registered by user (in their city)
orderby date.

① registered event by specific user



(upcoming) & city = user.city

} orderby

Result:

user	events	start_time
------	--------	------------

①

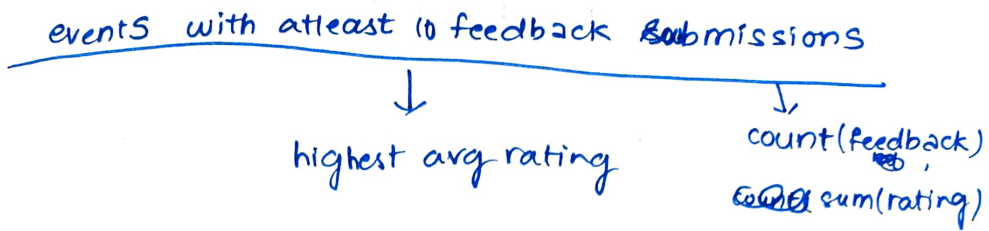
select user_id, event_id, ^{start_time}

where events.status = 'upcoming' and
events.city = user.city

groupby ^{user} user_id

orderby events.start_date.

② Top rated events: → events with atleast 10 feedbacks.



inner. { select event_id from feedback where
count(event_id) >= 10 } events with
atleast 10 feedback
forms



select event_id, avg(rating) from feedback
where event_id in (inner)
orderby avg(rating)

③ Retrieve users who have not registered for any events
in the last 90 days.

④ Peak session hours :

1, 2, 3, 4, 5

check

no. of sessions btw (10 AM - 12 PM) → for each event.

select count(session_id), event_id from sessions

where (starttime > 10 AM and endtime < 12 PM)

group by event_id

⑤ Most active cities : list the top 5 cities with the highest no. of distinct user registrations.

citywise → (distinct user's registered) count.

user table

registration table.

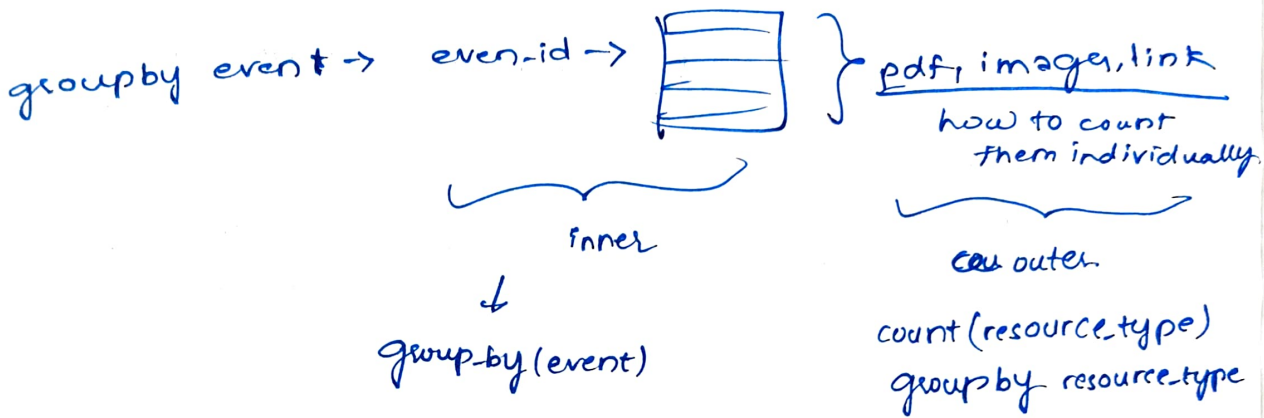
select city, count(distinct user_id) ^{as cnt} from users, registrations
group by city, order by cnt. DESC.
limit 5

① select u.city, count(distinct r.userid) as cnt
from users u join registrations r
on u.user_id = r.user_id
group by u.city
order by cnt DESC
limit 5;

⑥ no. of pdf, images, links uploaded → for each event.

~~select~~ count(pdf), count(images).

resource-type = { pdf, image, link }



event_id pdf-count, image-count, link-count.

select event-id,

sum(case when resource-type='pdf' then 1 else 0) as pdf-count

sum(case when resource-type='image' then 1 else 0) as image-count
link as link-count

from resources. groupby event-id.

⑦ Low feedback alerts:

users who have given rating < 3, comments, event-name.
events.
users, feedback.

select u.user-id, f.rating, f.comment, e.title
from users u join feedback f join events e
on u.user-id = f.user-id and
e.event-id = f.event-id

where f.rating < 3

user u join feedback on _____
join events e on _____

⑧ Sessions per upcoming event.

sessions events

No. of sessions per event
↓
upcoming

~~select e.title,~~

select count(session_id) (select e.event_id from events e where
from sessions s e.status = 'upcoming')
where s.event_id = e.event_id
group by s.event_id
upcoming events.
(event_id)

①



select e.event_id, e.title, count(s.session_id) as session-count
from events e

join sessions s on e.event_id = s.event_id

where e.status = 'upcoming'

group by e.event_id



⑨ Organizer event summary: (show no. of events created and their
status - upcoming, completed, cancelled
for each event organizer)

for each event-organiser → (in events table)

select ~~e.event_id~~ organizer, u.user_id, u.full_name.

sum(case when events.status = 'upcoming' then 1 else 0) as upcoming-count

①

"

as completed

"

as cancelled

from eventse group by e.organizer ;

left join users u

on e.organizer = u.user_id

(10) Feedback gap

Identify events that has registrations but not received any feedback.

$$\text{count}(\text{feedback}) = 0$$
~~select e.title, e.event_id~~

```
select e.title, e.event_id from events e feedback
```

where event_i in

```
select distinct event_id from events where
having count(registration_id) > 0
group by event_id
```

inner

and ^{having} count(feedback_id) = 0 groupby event_id

intersection: events

```
select event_id from registrations
group by event_id
having count (registration_id) > 0
and event_id not in (select
distinct event_id from feedback)
```

(11) daily new user count

daily new user count
find the no. of users who registered each day in last 7 days.

8.

$$\text{select count(*) count (distinct } \overbrace{\text{registration_date in (---,---)}}^{\text{register}} \text{)} \text{ } = 7$$

$$\text{group by user_id from registrations.}$$

```
select registration_date, count(user_id) from users
where registration_date >= curdate() - interval 7 days
group by registration_date
order by registration_date.
```

⑫ Events with maximum sessions.

select e.event_id, count(session_id) from
events e join sessions s on e.event_id = s.event_id

① group by e.event_id

order by sc. DESC

all events → no. of sessions in
descending order.

keep only max:

↓
(i)

having count(s.session_id) =

select max(session_count) from (select count(*) as
session_count from
sessions group by
event_id)

↓ (or)
(ii)

limit 1;

⑬ Avg. rating per city: Calculate avg feedback rating of
events conducted in each city.

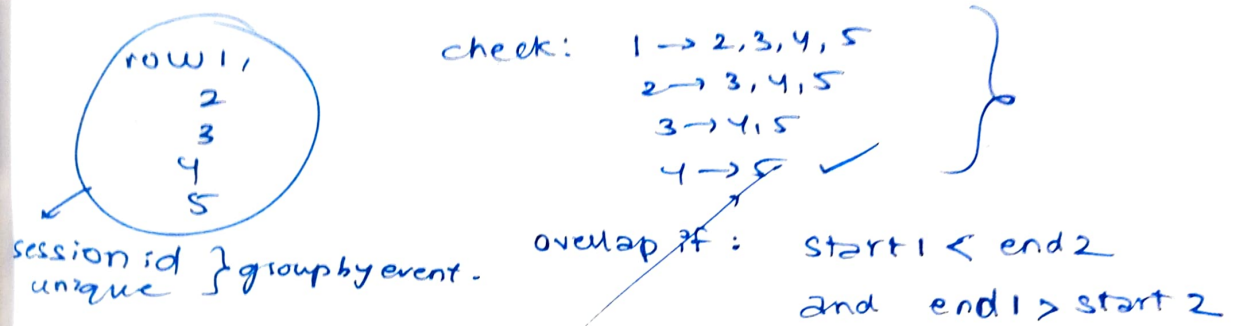
feedback, events.

① select e.city, avg(f.rating) as avg-feedback from
events e join feedback f on e.event_id = f.event_id
group by e.city.

⑭ Most registered events: List top 3 events based on the total
no. of user registrations.

select event_id count(user_id) as no. of users from registrations
group by event_id
order by no. of users DESC
Limit 3

- 5) Event session conflict : Identify overlapping sessions within the same event (i.e., session start & end times that conflict)



overlap → self join ★

~~select session~~ select s1.event_id, s1.session_id, s2.session_id
from sessions s1 join sessions s2
★ on s1.event_id = s2.event_id
AND s1.session_id < s2.session_id
AND s1.start_time < s2.end_time
and s1.end_time > s2.start_time.

eliminate duplicate comparisons. ←

Session 1 Session 2

curdate() - interval 30 day

- 16) Unregistered active users : Find users who created account in last 30 days but havenot registered for any events.

select u.user_id, u.full_name from users u
join registrations r on u.user_id = r.user_id
where u.registration_date >= curdate() - INTERVAL 30 day
AND
u.user_id not in (select distinct user_id from registrations)

select user_id, full_name from users where
u.registration_date >= curdate() - INTERVAL 30 day
and user_id not in (select distinct user_id from registrations)

17) Multi-session speakers

Identify speakers who are handling more than one session across all events.

```
select speak_name, count(session_id) as ns from sessions
where ns > 1 and count(session_id) > 1
group by speaker_name.
having count(session_id) > 1
```

18) List all events that do not have any resources uploaded

```
select e.event_id, e.title from events e join resources r
on e.event_id = r.event_id
X where e.event_id not in (select event_id from resources.registrations)
```

```
select e.event_id, e.title from events e
left join resources r on e.event_id = r.event_id
where r.resource_id is NULL;
```

left join

19) For completed events, show total registrations & average feedback.

↓
events.

↓
registrations

↓
feedback

```
select e.event_id, e.title, count(distinct registration_id), avg(f.rating)
from events e join registration on e.event_id = r.event_id
join feedback f on f.event_id = e.event_id
where e.status = 'completed'
group by e.event_id
```

✓

suggestion: use left join

20) User engagement index

For each user, calculate how many events they attended & how many feedbacks they submitted.

```
select u.user_id, u.full_name, count(r.event_id), count(f.event_id)
from users u join registration r on u.event_id = r.event_id
join feedback f on registration.event_id = u.user_id = f.user_id
group by u.user_id
```

21) Top feedback providers: list top 5 users who have submitted entries.

```
select u.user_id, u.full_name, count(distinct feedback_id)
from user u join feedback f on u.user_id = f.user_id
group by u.user_id.
ORDER BY count(f.feed_id) DESC
LIMIT 5
```

22) Duplicate registrations check:

Detect if a user has been registered more than once for the same event.

```
select u.user_id, u.full_name from users u join registrations r
on u.user_id = r.user_id
where
select user_id, event_id, count(*) as reg_count
from registrations
group by user_id, event_id
having count(*) > 1;
```

23) Registration trend

Show a month-wise registration count trend over the past 12 months.

group by month

```
select year(registration_date) as yr  
       month(registration_date) as mn  
       count(*) as registration_cnt
```

from registrations

where registration_date >= curdate() - INTERVAL 12 MONTH

group by year(registration_date), month(registration_date)

order by yr, mn

24) Average session duration per event

Compute avg duration (in min) of sessions in each event.

duration of session → $\frac{\text{TIMESTAMPDIFF}(\text{MINUTE}, \text{start_time}, \text{end_time})}{\text{count}}$

select event_id, Avg(($\downarrow$)) as avg-duration

from sessions

group by event_id

25) List all events that currently have no sessions scheduled under them.

select e.event_id, e.title from events e

left join sessions s on e.event_id = s.event_id

where s.session_id is NULL.